

FastAPI · Architecture · Gestion des erreurs · Logging · Debugging

Objectifs

À la fin de cette séance, vous serez capables de :

- comprendre pourquoi une API doit être structurée ;
- identifier les responsabilités des différentes parties d'une application FastAPI ;
- transformer une API contenue dans un seul fichier en plusieurs modules ;
- utiliser `APIRouter` ;
- séparer les routes, les modèles et la logique métier ;
- gérer correctement les erreurs HTTP ;
- utiliser `HTTPException` ;
- comprendre la différence entre erreurs 4xx et 5xx ;
- mettre en place des logs avec le module `logging` de Python ;
- utiliser les logs pour comprendre le comportement d'une API ;
- adopter une démarche systématique de debugging.

1. Point de départ : notre API Escape Game

Lors de la séance précédente, nous avons construit une API permettant de gérer les joueurs d'un escape game.

Notre application possède déjà plusieurs fonctionnalités :

```
GET    /players
GET    /players/{player_id}
POST   /players
PUT    /players/{player_id}
DELETE /players/{player_id}
```

Nous avons également utilisé :

- FastAPI ;
- Pydantic ;
- la validation automatique ;
- `HTTPException` ;
- une liste Python comme fausse base de données ;
- un compteur d'identifiants.

Nous allons repartir directement de cette application.

2. Notre fichier actuel

Notre API est actuellement contenue dans un seul fichier :

```
exemple_7.py
```

Son organisation est approximativement la suivante :

```
exemple_7.py

FastAPI

route GET /

données PLAYERS

compteur NEXT_PLAYER_ID

GET /players

GET /players/{player_id}

modèle Player

POST /players

PUT /players/{player_id}

DELETE /players/{player_id}
```

Pour une petite démonstration, cette organisation fonctionne parfaitement.

Le problème apparaît lorsque l'application commence à grandir.

3. Quand le fichier devient trop gros

Imaginons maintenant que notre API possède :

```
50 routes
20 modèles
30 règles métier
```

```
authentification
base de données
logs
gestion des erreurs
tests
```

Nous pourrions rapidement nous retrouver avec `main.py` contenant plusieurs milliers de lignes.

Le code peut toujours fonctionner. Mais il devient progressivement :

- difficile à lire ;
- difficile à comprendre ;
- difficile à modifier ;
- difficile à tester ;
- difficile à maintenir.

4. Le problème n'est pas FastAPI

```
@app.get('/players')
def get_players():
    ...
    # Organisation parfaitement valable
```

Le problème vient du fait que nous mélangeons plusieurs responsabilités dans le même fichier.

```
HTTP
+
validation
+
modèles
+
données
+
logique métier
+
gestion des erreurs
```

5. Une première règle d'architecture

Une règle très importante en développement logiciel est :

Une partie du programme doit avoir une responsabilité clairement identifiable.

Nous allons donc séparer notre application.

Application

- Routes HTTP
- Modèles de données
- Logique métier
- Accès aux données

6. Notre nouvelle architecture

Nous allons transformer notre exemple en :

```
game_api/  
  
  app/  
    __init__.py  
    main.py  
  
    models/  
      __init__.py  
      player.py  
  
    routers/  
      __init__.py  
      players.py  
  
    services/  
      __init__.py  
      player_service.py  
  
requirements.txt
```

Cette organisation peut sembler plus complexe. Mais chaque fichier possède maintenant une responsabilité claire.

7. Le rôle de `main.py`

Le fichier `app/main.py` sera responsable du démarrage et de la configuration de notre application FastAPI.

Il ne contiendra plus toute la logique de l'application. Son rôle sera notamment de :

- créer l'application FastAPI ;
- enregistrer les routers ;
- configurer l'application.

8. Le rôle des routers

Le dossier `app/routers/` contiendra les endpoints HTTP.

Par exemple :

```
routers/  
  players.py
```

contiendra :

```
GET    /players  
GET    /players/{player_id}  
POST   /players  
PUT    /players/{player_id}  
DELETE /players/{player_id}
```

Le router s'occupe donc principalement de la communication HTTP.

9. Le rôle des services

Le dossier `app/services/` contiendra la logique métier.

Par exemple :

```
services/  
  player_service.py
```

pourra s'occuper de :

- rechercher un joueur ;

- créer un joueur ;
- modifier un joueur ;
- supprimer un joueur.

Plus tard, cette couche pourra appliquer des règles métier plus complexes.

Les services s'occupent donc de la logique métier.

10. Le rôle des modèles

Le dossier `app/models/` contiendra les modèles de données.

Nous allons y placer :

```
player.py
```

avec nos modèles Pydantic.

Les modèles s'occupent des modèles de données.

11. Première étape : créer le modèle Player

Créons :

```
app/models/player.py
```

Contenu :

```
from pydantic import BaseModel, Field

class PlayerCreate(BaseModel):
    name: str = Field(min_length=3, max_length=99)
    score: int = Field(ge=0, le=99999)
    level: int = Field(ge=1, le=99)

class Player(PlayerCreate):
    id: int
```

Nous avons maintenant deux modèles.

12. Pourquoi deux modèles ?

Lorsqu'un utilisateur crée un joueur, il ne doit pas fournir son identifiant.

La requête contient :

```
{
  "name": "Link",
  "score": 1800,
  "level": 20
}
```

L'identifiant est créé par notre application.

La réponse contient :

```
{
  "id": 43,
  "name": "Link",
  "score": 1800,
  "level": 20
}
```

Nous avons donc :

```
PlayerCreate
  ↓
données fournies par le client

Player
  ↓
données complètes avec id
```

13. Les données du service

Créons :

```
app/services/player_service.py
```

Nous allons commencer par déplacer notre liste `PLAYERS`.

```

import logging

from app.models.player import Player, PlayerCreate

logger = logging.getLogger(__name__)

players = [
    Player(
        id=1,
        name='Alice',
        score=1200,
        level=12,
    ),
    Player(
        id=2,
        name='Bob',
        score=950,
        level=9,
    ),
    Player(
        id=42,
        name='Zelda',
        score=3000,
        level=99,
    ),
]

next_player_id = max(
    player.id for player in players
) + 1

```

Nous avons déplacé les données dans le service.

14. Pourquoi conserver `next_player_id` ?

Dans notre exemple précédent, nous avons déjà constaté qu'il était préférable de ne pas utiliser simplement :

```
len(players) + 1
```

Imaginons :

```
id = 1  
id = 2  
id = 42
```

Si nous supprimons le joueur 2, la longueur de la liste devient :

```
2
```

mais l'identifiant 3 est disponible.

Notre compteur permet de conserver une séquence indépendante de la taille actuelle de la liste.

15. Récupérer tous les joueurs

Dans :

```
player_service.py
```

ajoutons :

```
def get_players() -> list[Player]:  
    logger.info('Retrieving all players')  
    return players
```

Le service possède maintenant une fonction permettant de récupérer les joueurs.

16. Récupérer un joueur

Ajoutons :

```
def get_player(player_id: int) -> Player | None:
    logger.info(
        'Searching for player %s',
        player_id,
    )

    for player in players:
        if player.id == player_id:
            logger.info(
                'Player %s found',
                player_id,
            )
            return player

    logger.warning(
        'Player %s not found',
        player_id,
    )

    return None
```

Remarquez le type de retour :

```
Player | None
```

Cela signifie :

La fonction retourne soit un `Player`, soit `None`.

17. Créer un joueur

Ajoutons :

```
def create_player(player_data: PlayerCreate) -> Player:
    global next_player_id

    player = Player(
        id=next_player_id,
        **player_data.model_dump(),
    )
```

```

players.append(player)

logger.info(
    'Player %s created',
    player.id,
)

next_player_id += 1

return player

```

La responsabilité de créer le joueur appartient maintenant au service.

18. Modifier un joueur

Ajoutons :

```

def update_player(
    player_id: int,
    player_data: PlayerCreate,
) -> Player | None:
    for index, player in enumerate(players):
        if player.id == player_id:
            updated_player = Player(
                id=player_id,
                **player_data.model_dump(),
            )

            players[index] = updated_player

            logger.info(
                'Player %s updated',
                player_id,
            )

            return updated_player

    logger.warning(
        'Player %s not found for update',
        player_id,
    )

```

```
return None
```

Le service sait maintenant modifier un joueur.

19. Supprimer un joueur

Enfin :

```
def delete_player(player_id: int) -> Player | None:
    for index, player in enumerate(players):
        if player.id == player_id:
            deleted_player = players.pop(index)

            logger.info(
                'Player %s deleted',
                player_id,
            )

            return deleted_player

    logger.warning(
        'Player %s not found for deletion',
        player_id,
    )

    return None
```

Notre service contient maintenant toute la logique nécessaire à la gestion des joueurs.

20. Notre service complet

Le fichier :

```
app/services/player_service.py
```

est maintenant :

```

import logging

from app.models.player import Player, PlayerCreate

logger = logging.getLogger(__name__)

players = [
    Player(
        id=1,
        name='Alice',
        score=1200,
        level=12,
    ),
    Player(
        id=2,
        name='Bob',
        score=950,
        level=9,
    ),
    Player(
        id=42,
        name='Zelda',
        score=3000,
        level=99,
    ),
]

next_player_id = max(
    player.id for player in players
) + 1

def get_players() -> list[Player]:
    logger.info('Retrieving all players')
    return players

def get_player(player_id: int) -> Player | None:
    logger.info(

```

```

        'Searching for player %s',
        player_id,
    )

    for player in players:
        if player.id == player_id:
            logger.info(
                'Player %s found',
                player_id,
            )
            return player

    logger.warning(
        'Player %s not found',
        player_id,
    )

    return None

def create_player(player_data: PlayerCreate) -> Player:
    global next_player_id

    player = Player(
        id=next_player_id,
        **player_data.model_dump(),
    )

    players.append(player)

    logger.info(
        'Player %s created',
        player.id,
    )

    next_player_id += 1

    return player

def update_player(
    player_id: int,

```

```

        player_data: PlayerCreate,
    ) -> Player | None:
        for index, player in enumerate(players):
            if player.id == player_id:
                updated_player = Player(
                    id=player_id,
                    **player_data.model_dump(),
                )

                players[index] = updated_player

                logger.info(
                    'Player %s updated',
                    player_id,
                )

                return updated_player

        logger.warning(
            'Player %s not found for update',
            player_id,
        )

        return None

def delete_player(player_id: int) -> Player | None:
    for index, player in enumerate(players):
        if player.id == player_id:
            deleted_player = players.pop(index)

            logger.info(
                'Player %s deleted',
                player_id,
            )

            return deleted_player

    logger.warning(
        'Player %s not found for deletion',
        player_id,
    )

```

```
return None
```

21. Que contient maintenant le service ?

Le service contient :

Données

```
players
next_player_id
```

Logique

```
get_players()
get_player()
create_player()
update_player()
delete_player()
```

Il ne connaît pas FastAPI.

Il ne contient pas :

```
@app.get(...)
```

ou :

```
HTTPException
```

C'est volontaire.

22. Créons maintenant le router

Créons :

```
app/routers/players.py
```

Commençons par les imports :

```
from fastapi import APIRouter, HTTPException

from app.models.player import Player, PlayerCreate
from app.services import player_service
```

Puis :

```
router = APIRouter(
    prefix='/players',
    tags=['players'],
)
```

23. La route GET /players

Ajoutons :

```
@router.get('', response_model=list[Player])
def list_players() -> list[Player]:
    return player_service.get_players()
```

Nous retrouvons exactement le comportement de notre ancienne API.

GET /players

list_players()

player_service.get_players()

24. La route GET /players/{player_id}

Ajoutons :

```
@router.get('/{player_id}', response_model=Player)
def get_player(player_id: int) -> Player:
    player = player_service.get_player(player_id)

    if player is None:
```

```
        raise HTTPException(  
            status_code=404,  
            detail='Player not found',  
        )  
  
    return player
```

Cette fois, nous allons prendre le temps de comprendre la séparation.

25. Où se trouve la recherche ?

La recherche :

```
player_service.get_player(player_id)
```

est réalisée dans le service.

Le router ne sait pas comment le joueur est recherché.

Il demande simplement :

Donne-moi le joueur correspondant à cet identifiant.

26. Où se trouve l'erreur HTTP ?

L'erreur :

```
raise HTTPException(  
    status_code=404,  
    detail='Player not found',  
)
```

est dans le router.

Pourquoi ?

Parce que 404 est une notion HTTP.

Le service, lui, peut simplement dire :

```
return None
```

Le router transforme ensuite cette situation métier en réponse HTTP.

27. Une distinction importante

Nous avons donc :

```
Service

    "Le joueur n'existe pas"
    ↓
    None

Router

    "Cette situation correspond à HTTP 404"
    ↓
    HTTPException
```

Cette distinction deviendra très importante lorsque nous introduirons une base de données.

28. La route POST

Ajoutons :

```
@router.post(
    '',
    response_model=Player,
    status_code=201,
)
def create_player(player: PlayerCreate) -> Player:
    return player_service.create_player(player)
```

La route reçoit :

```
PlayerCreate
```

Pydantic valide automatiquement les données.

Puis :

```
player_service.create_player(player)
```

prend le relais.

29. La route PUT

Ajoutons :

```
@router.put(
    '/{player_id}',
    response_model=Player,
)
def update_player(
    player_id: int,
    player: PlayerCreate,
) -> Player:
    updated_player = player_service.update_player(
        player_id,
        player,
    )

    if updated_player is None:
        raise HTTPException(
            status_code=404,
            detail='Player not found',
        )

    return updated_player
```

Nous retrouvons le même fonctionnement que dans notre fichier initial.

Mais la logique est maintenant séparée.

30. La route DELETE

Ajoutons :

```
@router.delete('/{player_id}')
def delete_player(player_id: int) -> dict:
    deleted_player = player_service.delete_player(
        player_id
    )

    if deleted_player is None:
        raise HTTPException(
            status_code=404,
```

```

        detail='Player not found',
    )

    return {
        'message': 'Player deleted',
        'player': deleted_player,
    }

```

31. Le router complet

Notre fichier :

app/routers/players.py

est maintenant :

```

from fastapi import APIRouter, HTTPException

from app.models.player import Player, PlayerCreate
from app.services import player_service

router = APIRouter(
    prefix='/players',
    tags=['players'],
)

@router.get('', response_model=list[Player])
def list_players() -> list[Player]:
    return player_service.get_players()

@router.get('/{player_id}', response_model=Player)
def get_player(player_id: int) -> Player:
    player = player_service.get_player(player_id)

    if player is None:
        raise HTTPException(
            status_code=404,
            detail='Player not found',

```

```

    )

    return player

@router.post(
    '',
    response_model=Player,
    status_code=201,
)
def create_player(player: PlayerCreate) -> Player:
    return player_service.create_player(player)

@router.put(
    '/{player_id}',
    response_model=Player,
)
def update_player(
    player_id: int,
    player: PlayerCreate,
) -> Player:
    updated_player = player_service.update_player(
        player_id,
        player,
    )

    if updated_player is None:
        raise HTTPException(
            status_code=404,
            detail='Player not found',
        )

    return updated_player

@router.delete('/{player_id}')
def delete_player(player_id: int) -> dict:
    deleted_player = player_service.delete_player(
        player_id
    )

```

```

if deleted_player is None:
    raise HTTPException(
        status_code=404,
        detail='Player not found',
    )

return {
    'message': 'Player deleted',
    'player': deleted_player,
}

```

32. Le rôle de APIRouter

Nous avons écrit :

```

router = APIRouter(
    prefix='/players',
    tags=['players'],
)

```

Cela signifie que ce router représente la ressource :

players

Toutes les routes du fichier sont donc regroupées.

```

players

GET    /players
GET    /players/{player_id}
POST   /players
PUT    /players/{player_id}
DELETE /players/{player_id}

```

33. Pourquoi le prefix ?

Nous avons :

```
prefix= '/players'
```

et :

```
@router.get('')
```

Le résultat est :

```
/players
```

De même :

```
@router.get('/{player_id}')
```

devient :

```
/players/{player_id}
```

Cela évite de répéter `/players` dans chaque route.

34. Pourquoi les tags ?

Nous avons :

```
tags=['players']
```

FastAPI utilise ces tags dans la documentation Swagger.

Dans :

```
/docs
```

les endpoints seront regroupés sous :

```
players
```

Cela devient particulièrement intéressant lorsque nous avons plusieurs routers.

35. Créons maintenant `main.py`

Le fichier :

`app/main.py`

reste très simple.

```
import logging

from fastapi import FastAPI

from app.routers.players import router as players_router

logging.basicConfig(level=logging.INFO)

app = FastAPI(
    title='Game API',
    description='API de démonstration pour un escape game',
    version='1.0.0',
)

app.include_router(players_router)

@app.get('/')
def home() -> dict:
    return {
        'message': 'Bienvenue sur l API pour mon super escape game!'
    }
```

36. Pourquoi `main.py` est-il si petit ?

Parce que nous avons déplacé les responsabilités.

Avant :

```
main.py
```

```
modèles  
données  
routes  
logique  
erreurs
```

Maintenant :

```
main.py
```

```
configuration de l'application
```

C'est précisément le but de la refactorisation.

37. Notre architecture finale

Nous avons maintenant :

```
game_api/
```

```
app/  
  __init__.py  
  main.py  
  
  models/  
    __init__.py  
    player.py  
  
  routers/  
    __init__.py  
    players.py  
  
  services/  
    __init__.py  
    player_service.py  
  
requirements.txt
```

38. Le fichier requirements.txt

Pour rendre le projet directement installable :

```
fastapi[standard]
```

Installation :

```
pip install -r requirements.txt
```

39. Lancer l'application

Depuis le dossier :

```
game_api/
```

lancer :

```
fastapi dev app/main.py
```

FastAPI démarre alors le serveur de développement.

L'API est accessible sur :

```
http://127.0.0.1:8000
```

40. Tester la route d'accueil

Ouvrir :

```
http://127.0.0.1:8000/
```

Résultat :

```
{  
  "message": "Bienvenue sur l API pour mon super escape game!"  
}
```

41. Tester les joueurs

Ouvrir :

```
http://127.0.0.1:8000/players
```

Nous obtenons notamment :

```
[
  {
    "id": 1,
    "name": "Alice",
    "score": 1200,
    "level": 12
  },
  {
    "id": 2,
    "name": "Bob",
    "score": 950,
    "level": 9
  },
  {
    "id": 42,
    "name": "Zelda",
    "score": 3000,
    "level": 99
  }
]
```

42. Tester un joueur

```
GET /players/1
```

Résultat :

```
{
  "id": 1,
  "name": "Alice",
  "score": 1200,
  "level": 12
}
```

43. Tester un joueur inexistant

Essayons :

```
GET /players/999
```

Nous obtenons :

```
404 Not Found
```

avec :

```
{  
  "detail": "Player not found"  
}
```

C'est maintenant une véritable réponse HTTP d'erreur.

44. Comparaison avec notre ancienne API

Dans notre première version, le joueur inexistant provoquait une réponse de type :

```
{  
  "error": "Player not found"  
}
```

mais avec un code HTTP qui pouvait rester :

```
200 OK
```

Ce comportement était ambigu.

Maintenant :

```
404 Not Found
```

indique explicitement au client que la ressource n'existe pas.

45. Tester la validation

Essayons :

```
GET /players/abc
```

Notre fonction attend :

```
player_id: int
```

FastAPI détecte donc automatiquement que :

```
abc
```

n'est pas un entier.

La requête est rejetée avant même d'arriver à notre service.

46. Comprendre le chemin de la requête

Pour :

```
GET /players/42
```

le chemin est :

```
Client
```

```
FastAPI
```

```
Router
```

```
player_service.get_player(42)
```

```
Player
```

Si le joueur n'existe pas :

```
Client
```

```
Router
```

```
Service
```

```
None
```

```
HTTPException(404)
```

```
Client
```

47. Les logs

Nous avons ajouté :

```
import logging
```

puis :

```
logger = logging.getLogger(__name__)
```

et par exemple :

```
logger.info(  
    'Searching for player %s',  
    player_id,  
)
```

Nous pouvons maintenant observer l'activité de notre application.

48. Les différents niveaux de logs

Python propose notamment :

Niveau	Signification
DEBUG	informations très détaillées
INFO	fonctionnement normal
WARNING	situation inhabituelle
ERROR	erreur
CRITICAL	erreur critique

Exemples :

```
logger.debug('Detailed information')
```

```
logger.info('Player created')
```

```
logger.warning('Player not found')
```

```
logger.error('Database connection failed')
```

49. Pourquoi utiliser un logger ?

On pourrait être tenté d'utiliser :

```
print('Player found')
```

Cela peut être pratique pour un premier test.

Mais dans une véritable application, nous préférons :

```
logger.info('Player %s found', player_id)
```

Le système de logging permet notamment de :

- filtrer les niveaux ;
- configurer les sorties ;
- conserver des traces ;
- ajouter des timestamps ;
- adapter le comportement selon l'environnement.

50. print ou logging ?

Pour du debugging rapide :

```
print(variable)
```

peut être utile.

Pour une application :

```
logger.info(...)
logger.warning(...)
logger.error(...)
```

est préférable.

On peut retenir :

```
print
↓
debug ponctuel

logging
↓
observation de l'application
```

51. Ajouter du contexte aux logs

Comparez :

```
logger.error('Error')
```

avec :

```
logger.error(
    'Unable to retrieve player %s',
    player_id,
)
```

Le deuxième message est beaucoup plus utile.

Un bon log doit fournir suffisamment de contexte pour comprendre ce qui s'est passé.

52. Exemple de trace

Lors d'une requête :

```
GET /players/42
```

nous pouvons obtenir :

```
INFO Searching for player 42  
INFO Player 42 found
```

Pour :

```
GET /players/999
```

nous pouvons obtenir :

```
INFO Searching for player 999  
WARNING Player 999 not found
```

Les logs racontent donc ce qui s'est passé.

53. Les erreurs HTTP

Il est important de distinguer les familles de codes HTTP.

```
2xx  
    succès  
  
4xx  
    problème lié à la requête du client  
  
5xx  
    problème côté serveur
```

Quelques exemples :

```
200 OK
201 Created
204 No Content

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
422 Unprocessable Entity

500 Internal Server Error
```

54. Une ressource inexistante

Si :

```
GET /players/999
```

et que le joueur n'existe pas :

```
404 Not Found
```

est approprié.

Dans notre route :

```
if player is None:
    raise HTTPException(
        status_code=404,
        detail='Player not found',
    )
```

55. Une donnée invalide

Si nous envoyons :

```
{
    "name": "A",
    "score": -10,
    "level": 0
}
```

notre modèle :

```
class PlayerCreate(BaseModel):
    name: str = Field(min_length=3, max_length=99)
    score: int = Field(ge=0, le=99999)
    level: int = Field(ge=1, le=99)
```

refuse automatiquement les données.

Nous n'avons donc pas besoin d'écrire manuellement :

```
if len(name) < 3:
    ...
```

56. Une erreur inattendue

Imaginons maintenant :

```
result = 10 / 0
```

Python provoquera :

```
ZeroDivisionError
```

Cette erreur n'est pas une situation métier normale.

Il s'agit d'un problème côté serveur.

Nous devons donc pouvoir la diagnostiquer.

57. Ne pas masquer les exceptions

Évitez :

```
try:
    result = perform_operation()

except Exception:
    return {
        'error': 'Something went wrong'
    }
```

Cette approche masque l'erreur réelle.

Nous perdons alors des informations importantes pour le debugging.

58. `logger.exception()`

Lorsque nous sommes dans un `except`, nous pouvons utiliser :

```
try:
    result = perform_operation()

except Exception:
    logger.exception(
        'Unexpected error during operation'
    )
    raise
```

`logger.exception()` enregistre également la trace de l'exception.

La trace permet notamment de connaître :

- le type de l'erreur ;
- le fichier ;
- la ligne concernée ;
- la chaîne d'appels.

59. Exemple de debugging

Imaginons que :

```
GET /players/42
```

retourne :

```
500 Internal Server Error
```

Nous consultons les logs :

```
INFO Searching for player 42
ERROR Database connection failed
```

Nous pouvons alors formuler une hypothèse beaucoup plus précise :



60. Une méthode systématique de debugging

Lorsqu'une API ne fonctionne pas :

Étape 1 — Reproduire

Exemple :

```
GET /players/42
```

Étape 2 — Regarder le code HTTP

```
200 ?  
404 ?  
422 ?  
500 ?
```

Étape 3 — Lire le message d'erreur

Étape 4 — Consulter les logs

Étape 5 — Identifier la couche responsable

```
Router ?  
Service ?  
Données ?
```

Étape 6 — Corriger

Étape 7 — Reproduire le problème

61. Une règle d'or

Ne cherchez pas à résoudre un bug en modifiant du code au hasard.

Essayez plutôt de répondre successivement à :

```
Qu'est-ce que j'ai demandé ?  
↓  
Qu'est-ce que j'ai obtenu ?  
↓  
Quel est le code HTTP ?  
↓  
Que disent les logs ?  
↓  
À quelle étape le comportement devient-il incorrect ?
```

C'est une démarche de debugging.

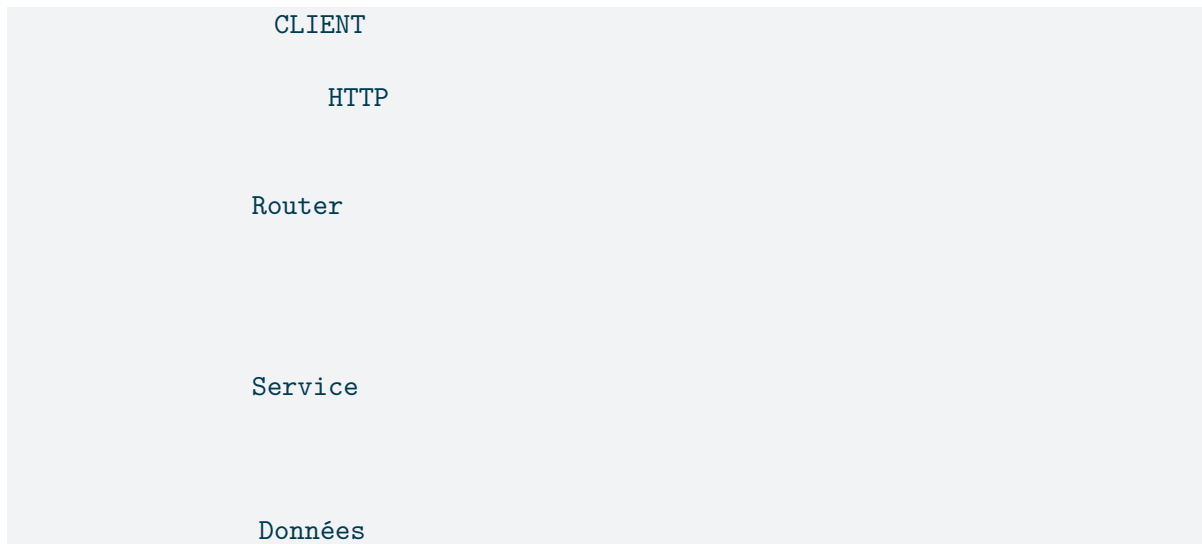
62. Responsabilités des différentes couches

Nous avons maintenant une séparation claire.

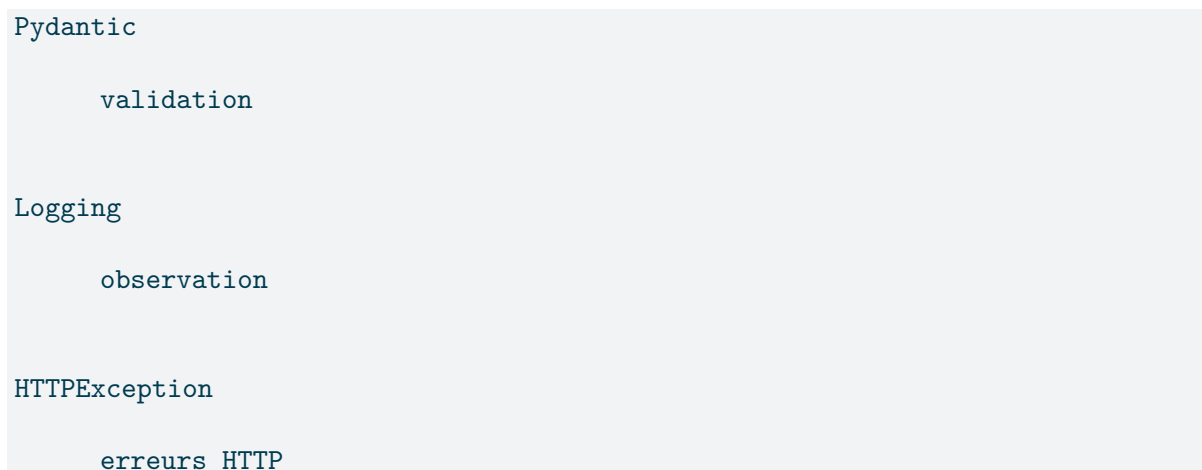
Élément	Responsabilité
<code>main.py</code>	configuration de l'application
<code>routers/</code>	HTTP et endpoints
<code>models/</code>	structures et validation des données
<code>services/</code>	logique métier
<code>logging</code>	observation de l'application

63. Le flux complet

Notre application fonctionne maintenant ainsi :



Avec :



64. Pourquoi cette architecture va devenir intéressante ?

Aujourd'hui, notre service utilise :

```
players = [...]
```

Il s'agit d'une liste Python.

Mais demain, nous voulons probablement utiliser :

PostgreSQL

Nous ne voulons pas devoir modifier toutes les routes.

Nous voulons pouvoir passer progressivement de :

```
Router
  ↓
Service
  ↓
liste Python
```

à :

```
Router
  ↓
Service
  ↓
Repository
  ↓
SQLAlchemy
  ↓
PostgreSQL
```

65. Le futur repository

Nous pourrions alors créer :

```
app/

  routers/
    players.py

  services/
    player_service.py

  repositories/
    player_repository.py

  models/
    player.py
```

```
database/
```

Le repository sera responsable de l'accès aux données.

66. Router, service et repository

On peut retenir :

Router

"Comment communiquer avec le client ?"

Service

"Quelles sont les règles de l'application ?"

Repository

"Comment récupérer/enregistrer les données ?"

Database

67. Exemple concret

Imaginons la règle :

Un joueur ne peut pas dépasser le niveau 99.

La validation de cette donnée peut être exprimée avec Pydantic :

```
level: int = Field(ge=1, le=99)
```

Mais imaginons une règle plus complexe :

Un joueur ne peut augmenter son niveau que s'il possède suffisamment de points.

Cette règle relève davantage de la logique métier.

Elle pourrait donc être gérée dans :

Service

68. Autre exemple

La question :

Où se trouve le joueur 42 dans PostgreSQL ?

relève de :

Repository

La question :

Que doit retourner l'API si le joueur n'existe pas ?

relève de la couche HTTP :

Router

Nous avons donc :

```
Recherche
  → Repository

Règle métier
  → Service

Réponse HTTP
  → Router
```

69. Niveau 1 — Guidé

Exercice

Ajoutez une route permettant de rechercher uniquement les joueurs ayant un niveau supérieur ou égal à une valeur donnée.

Exemple :

```
GET /players/level/10
```

doit retourner les joueurs ayant :

```
level >= 10
```

Indication

Commencez par ajouter une fonction dans :

```
player_service.py
```

Puis créez la route dans :

```
players.py
```

Objectif

Comprendre le chemin :

```
HTTP
↓
Router
↓
Service
↓
Données
```

70. Niveau 2 — Réflexion

Ajoutez une route :

```
GET /players/search/{name}
```

permettant de rechercher un joueur à partir de son nom.

Exemple :

```
GET /players/search/Alice
```

La recherche doit être insensible à la casse.

Ainsi :

```
Alice  
alice  
ALICE
```

doivent permettre de retrouver le même joueur.

Question

Dans quelle couche doit se trouver la logique de comparaison des noms ?

71. Niveau 3 — Challenge

Imaginez maintenant que plusieurs joueurs puissent avoir le même nom.

La route :

```
GET /players/search/Alice
```

ne doit donc plus nécessairement retourner un seul joueur.

Elle pourrait retourner :

```
[  
  {  
    "id": 1,  
    "name": "Alice",  
    "score": 1200,  
    "level": 12  
  },  
  {  
    "id": 45,  
    "name": "Alice",  
    "score": 2100,  
    "level": 20  
  }  
]
```

Objectif

Adapter :

- le service ;
- le router ;
- le modèle de réponse.

72. Niveau 1 — Gestion des erreurs

Modifiez une route afin qu'elle retourne :

```
404 Not Found
```

lorsqu'un joueur demandé n'existe pas.

Testez :

```
GET /players/999
```

Vous devez obtenir :

```
{
  "detail": "Player not found"
}
```

73. Niveau 2 — Logging

Ajoutez des logs permettant de suivre la création d'un joueur.

Lorsqu'un joueur est créé, vous devez obtenir une information comparable à :

```
INFO Player 43 created
```

Lorsqu'une recherche échoue :

```
WARNING Player 999 not found
```

74. Niveau 3 — Debugging

Ajoutez volontairement une erreur dans le service.

Par exemple :

```
result = 10 / 0
```

Appelez ensuite la route concernée.

Observez :

```
HTTP 500
```

puis consultez les logs.

Utilisez :

```
logger.exception(...)
```

pour obtenir la trace complète de l'erreur.

75. Vérification finale

À la fin de cette séance, notre projet doit avoir cette structure :

```
game_api/  
  
  app/  
    __init__.py  
    main.py  
  
    models/  
      __init__.py  
      player.py  
  
    routers/  
      __init__.py  
      players.py  
  
    services/  
      __init__.py
```

```
player_service.py
```

```
requirements.txt
```

76. Checklist

Organisation

- ☐ Les routes sont dans `routes/`.
- ☐ Les modèles sont dans `models/`.
- ☐ La logique métier est dans `services/`.
- ☐ `main.py` reste simple.
- ☐ Les responsabilités sont clairement séparées.

HTTP

- ☐ GET fonctionne.
- ☐ POST fonctionne.
- ☐ PUT fonctionne.
- ☐ DELETE fonctionne.
- ☐ Les ressources inexistantes retournent 404.

Validation

- ☐ Les modèles Pydantic sont utilisés.
- ☐ Les contraintes sont définies avec `Field`.
- ☐ Les données invalides sont rejetées automatiquement.

Logging

- ☐ Les opérations importantes sont journalisées.
- ☐ Les recherches sont visibles dans les logs.
- ☐ Les ressources inexistantes génèrent un `WARNING`.
- ☐ Les exceptions inattendues sont journalisées.

77. Ce que nous avons gagné

Avant :

```
exemple_7.py  
  
120 lignes  
routes  
données  
modèles  
logique métier  
erreurs
```

Après :

```
app/  
  
main.py  
models/  
routers/  
services/
```

L'application fait toujours la même chose.

Mais son organisation est maintenant beaucoup plus claire.

78. Le principe essentiel

La structuration d'une application ne consiste pas à ajouter des fichiers simplement pour avoir plus de fichiers.

L'objectif est de pouvoir répondre facilement à la question :

Où dois-je modifier le code si je veux changer ce comportement ?

Par exemple :

```
Modifier une route HTTP  
    ↓  
routers/  
  
Modifier une structure de données
```

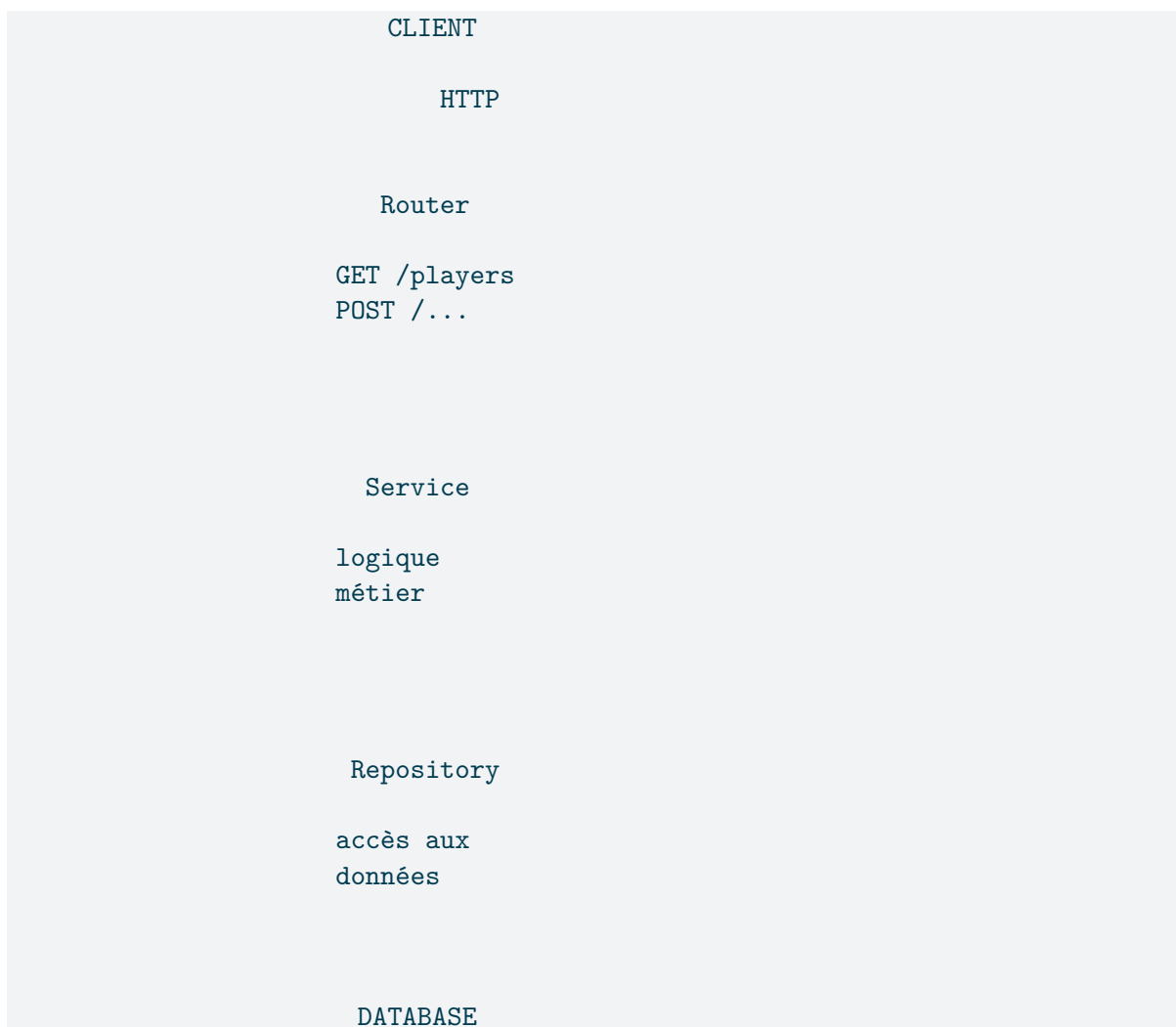
↓
models/

Modifier une règle métier

↓
services/

79. Architecture cible

Nous avons maintenant les bases d'une architecture backend :



Avec :

Pydantic

validation des entrées
validation des sorties

Logging

observation et debugging

80. Transition vers la suite

Notre API utilise encore :

```
players = [...]
```

Les données sont donc uniquement conservées en mémoire.

Si nous arrêtons le serveur :

Serveur arrêté

↓

Données perdues

Nous avons maintenant une architecture suffisamment propre pour introduire une véritable couche d'accès aux données.

La prochaine étape sera donc :

FastAPI

↓

Router

↓

Service

↓

Repository

↓

SQLAlchemy

↓

PostgreSQL

Nous allons ainsi passer progressivement d'une API de démonstration à une véritable application backend persistante.